- Spring provides a Spring REST Docs project (https://spring.io/projects/spring-restdocs) that helps you document REST APIs via test-driven development (TDD), resulting in high-quality and maintainable documentation. The resulting documentation is aimed at people, relying on formats like Asciidoc or Markdown. If you'd like to obtain an OpenAPI representation as well, you can check the restdocs-api-spec community-driven project to add OpenAPI support to Spring REST Docs (https://github.com/ePages-de/restdocs-api-spec).
- The springdoc-openapi community-driven project helps automate the generation of API documentation according to the OpenAPI 3 format (https://springdoc.org).

The contract is established through the REST API, so let's move on and look at the business logic. The solution is centered around three concepts:

- *Entity*—An entity represents the noun in a domain, such as "book."
- *Service*—A service defines the use cases for the domain. For example, "adding a book to the catalog."
- *Repository*—A repository is an abstraction to let the domain layer access data independently from its source.

Let's start with the domain entity.

DEFINING THE DOMAIN ENTITY

The REST API defined in table 3.1 should make it possible to operate on books. That's the *domain entity*. In the Catalog Service project, create a new `com.polarbookshop.catalogservice.domain` package for the business logic, and create a `Book` Java record to represent the domain entity.